**Gestión de pedidos de cafetería**

Crea una aplicación WPF denominada Gestión de pedidos de cafetería. El objetivo principal de la práctica es integrar los contenidos trabajados sobre clases y objetos, ObservableCollection, ListView, Binding, SelectedItem, DataContext y comunicación entre ventanas.

La aplicación permitirá registrar pedidos, mantener separados los pedidos pendientes y los pedidos preparados y trasladar un mismo objeto de una colección a otra cuando se complete su preparación. Solo se utilizará una clase de datos en toda la práctica.

1. **Estructura general.**

   La aplicación estará formada por una ventana principal y una segunda ventana destinada a confirmar la preparación de un pedido seleccionado.

   La ventana principal deberá incluir una zona de registro, una zona de acciones y dos zonas de listado: pedidos pendientes y pedidos preparados. No se proporciona una estructura interna exacta: deberás seleccionar y combinar los paneles que consideres adecuados para conseguir una interfaz clara y coherente.

2. **Clase Pedido.**

   Crea una única clase de datos denominada Pedido. Cada objeto representará un pedido concreto y deberá almacenar, como mínimo, las siguientes propiedades:

   - Nombre del cliente.
   - Producto.
   - Cantidad.
   - Tipo de recogida.
   - Observaciones.

   Utiliza los tipos de datos que consideres adecuados para cada propiedad. Las observaciones podrán quedar vacías.

3. **Formulario de registro.**

   La ventana principal deberá permitir introducir los datos necesarios para registrar un nuevo pedido. Incluye, como mínimo:

   - Un TextBox para introducir el nombre del cliente.
   - Un ComboBox para seleccionar el producto: Café, Té, Bocadillo o Repostería.
   - Un control adecuado para introducir la cantidad.
   - Controles adecuados para seleccionar el tipo de recogida: En local o Para llevar.
   - Un TextBox para introducir observaciones. Este control deberá ser adecuado para escribir texto de varias líneas.

4. **Validación de los datos.**

   Antes de registrar un pedido deberán comprobarse, al menos, las siguientes condiciones:

   - El nombre del cliente no puede estar vacío.
   - Debe haberse seleccionado un producto.
   - La cantidad debe ser un número válido y mayor que cero.
   - Debe haberse seleccionado un tipo de recogida.

   Si alguna condición no se cumple, deberá mostrarse un mensaje adecuado y el pedido no deberá registrarse.

5. **Colecciones de pedidos.**

   La aplicación utilizará dos colecciones del tipo ObservableCollection<Pedido>: una para los pedidos pendientes y otra para los pedidos preparados.

Cada colección deberá asociarse mediante ItemsSource a su ListView correspondiente. Cuando un pedido se añada o elimine de una colección, el listado deberá actualizarse automáticamente. No deberá utilizarse Items.Refresh() para conseguir estas actualizaciones.

6. **Registro de pedidos.**

   Incluye un botón denominado Registrar pedido. Cuando los datos sean correctos, este botón deberá obtener la información del formulario, crear un nuevo objeto Pedido y añadirlo exclusivamente a la colección de pedidos pendientes.

   El nuevo pedido deberá aparecer automáticamente en el listado de pendientes. El botón Registrar pedido será el único encargado de crear nuevos objetos Pedido.

7. **Listado de pedidos.**

   Los pedidos se mostrarán en dos ListView utilizando una vista con columnas. Ambos listados deberán mostrar, como mínimo: Cliente, Producto, Cantidad y Tipo de recogida.

   Los valores de las columnas deberán obtenerse mediante Binding a las propiedades de Pedido. Las observaciones no tienen que mostrarse en los ListView; se consultarán en la segunda ventana.

8. **Preparar un pedido.**

   Añade un botón denominado Preparar pedido. Al pulsarlo deberá comprobarse que existe un pedido seleccionado en el ListView de pedidos pendientes.

   Si existe una selección válida, deberá recuperarse mediante SelectedItem el objeto Pedido correspondiente y enviarlo a la segunda ventana mediante su constructor. No deberá crearse un nuevo objeto para esta operación.

9. **Ventana de preparación y DataContext.**

   Crea una segunda ventana denominada, por ejemplo, VentanaPreparacion.xaml. La ventana recibirá mediante su constructor el objeto Pedido seleccionado.

   El objeto recibido deberá establecerse como DataContext de la ventana. Los datos del pedido se mostrarán mediante Binding, incluyendo el nombre del cliente, producto, cantidad, tipo de recogida y observaciones.

   La ventana deberá incluir dos botones: Confirmar preparación y Cancelar.

10. **Confirmar o cancelar la preparación.**

    Si el usuario pulsa Confirmar preparación, la segunda ventana deberá cerrarse indicando que la operación ha sido aceptada mediante el mecanismo trabajado en clase.

    Si pulsa Cancelar, la ventana deberá cerrarse indicando que la operación no ha sido aceptada. En este caso el pedido deberá permanecer en la colección de pendientes sin ningún cambio.

11. **Mover el pedido entre colecciones.**

    Cuando la ventana principal reciba la confirmación de que la preparación ha sido aceptada, deberá trasladar el pedido seleccionado desde la colección de pendientes hasta la colección de preparados.

    Para realizar esta operación se trabajará con el mismo objeto Pedido: deberá eliminarse de la colección de pendientes y añadirse a la colección de preparados. No deberá crearse una copia ni un nuevo objeto Pedido.

    El pedido deberá desaparecer automáticamente del primer ListView y aparecer automáticamente en el segundo, sin utilizar Items.Refresh().

12. **Devolver un pedido a pendientes.**

    Añade un botón denominado Devolver a pendientes asociado al listado de pedidos preparados. Si existe un pedido seleccionado, deberá trasladarse nuevamente a la colección de pendientes utilizando el mismo objeto.

    Esta operación no necesita abrir una nueva ventana. Si no existe una selección válida, deberá mostrarse un mensaje adecuado.

13. **Contadores.**

    La ventana principal deberá mostrar en todo momento el número de pedidos pendientes y el número de pedidos preparados.

    Los dos contadores deberán actualizarse después de registrar un pedido y después de mover un pedido de una colección a otra, utilizando la información disponible en las propias colecciones.

14. **Funcionamiento esperado.**

    El flujo principal deberá permitir registrar varios pedidos, mostrarlos automáticamente como pendientes, seleccionar uno de ellos y abrir la ventana de preparación. Si el usuario confirma la operación, el mismo objeto deberá pasar al listado de preparados; si cancela, deberá permanecer en pendientes.

    También deberá ser posible devolver un pedido preparado al listado de pendientes y comprobar que ambos ListView y sus contadores reflejan inmediatamente los cambios.

15. **Condiciones finales.**

    La práctica deberá aplicar correctamente una única clase Pedido, creación de objetos, dos ObservableCollection<Pedido>, dos ListView con GridView, ItemsSource, Binding, SelectedItem, validaciones, dos ventanas WPF, paso del objeto mediante constructor, DataContext, ShowDialog(), DialogResult y traslado del mismo objeto entre colecciones mediante operaciones de adición y eliminación.

    No deberá utilizarse Items.Refresh() para actualizar los ListView después de añadir, eliminar o trasladar pedidos. La interfaz deberá mantener una distribución clara y coherente y los nombres de controles, variables, propiedades y manejadores deberán permitir identificar fácilmente su finalidad.